

Fiche technique — analyse de survie colon

Document de travail **privé** : à ne pas verser dans le dépôt public. Sa fonction est de te permettre de défendre chaque ligne du projet à l'oral, pas d'expliquer le code à un lecteur du README.

Ordre suivi : celui du fichier 01_survival_analysis.Rmd après application des six modifications.

1. Chunk setup — paramètres globaux du rendu

```
r

knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE,
  fig.path = here::here("figures", ""),
  fig.width = 8, fig.height = 6)
```

Argument	Rôle	Ce qui se passerait sans lui
echo = TRUE	Le code apparaît dans le rapport HTML	Le rapport ne montrerait que les résultats — inacceptable pour une démonstration méthodologique, où le code est le livrable
warning = FALSE	Masque les avertissements R	Le rapport serait pollué par les messages de survminer . Contrepartie réelle : masque aussi les avertissements légitimes. À désactiver ponctuellement pendant le développement
message = FALSE	Masque les messages de chargement de paquets	Idem
fig.path	Répertoire d'écriture des figures	Les PNG iraient dans un dossier _files temporaire et le README ne pourrait pas les référencer
fig.width/height	Dimensions en pouces	Figures aux proportions par défaut, souvent trop étroites pour une table de risque

Pourquoi here::here() et non "../figures/". here() remonte à la racine du projet (repérée par le .Rproj) et renvoie un chemin absolu. Les figures atterrissent donc toujours dans racine/figures/, quel que soit le répertoire de travail du knitr. Un chemin relatif ne fonctionne que si le knitr s'exécute depuis analysis/ — ce qui cesse d'être vrai dès qu'on règle knit_root_dir sur la racine du projet.

Le chemin absolu ne contamine pas le HTML livré : `html_document` a `self_contained = TRUE` par défaut, donc pandoc encode les images en base64 au moment du rendu.

2. Chunk `packages` — dépendances

```
r

library(survival)  # Surv(), survfit(), coxph(), cox.zph(), survdiff()
library(survminer) # ggsvrplot(), ggforest(), ggcoxzph()
library(dplyr)     # manipulation de données
library(ggplot2)   # graphique descriptif
library(here)      # résolution des chemins
```

`survival` est le paquet de référence historique en analyse de survie sous R (Terry Therneau, Mayo Clinic) — c'est aussi lui qui distribue le jeu `colon`. `survminer` n'apporte aucune méthode statistique : ce sont des habillages `ggplot2` autour des objets produits par `survival`.

À savoir dire : aucune de mes estimations ne vient de `survminer`. Il ne fait que tracer ce que `survival` a calculé.

3. Chunk `data` — construction du jeu d'analyse

```
r

data(colon)
```

Charge le jeu depuis le paquet. **Conséquence pour la reproductibilité :** aucun fichier de données n'est versionné, et `renv.lock` fige la version du paquet, donc la version exacte des données. C'est plus robuste qu'un CSV commité.

```
r

df <- colon %>%
  filter(etype == 2) %>%
```

La ligne la plus importante du projet. `colon` contient deux lignes par patient : `etype == 1` pour la récurrence, `etype == 2` pour le décès. 1 858 lignes pour 929 individus.

Sans ce filtre, chaque patient est traité comme deux observations indépendantes.

Conséquences :

- l'effectif apparent double,
- les erreurs standards sont sous-estimées (on croit avoir deux fois plus d'information qu'en réalité),
- les p-values sont artificiellement petites,

- **aucun avertissement n'est émis** — le modèle tourne et produit des estimations plausibles.

`etype == 2` définit l'endpoint **survie globale**. C'est un choix, pas une évidence : `etype == 1` donnerait la survie sans récurrence, avec un problème de risque compétitif (voir § 13).

```
r  
  
mutate(  
  rx = factor(rx, levels = c("Obs", "Lev", "Lev+5FU")),
```

Fixe l'ordre des niveaux, donc le niveau de référence. `Obs` en premier $\Rightarrow$ tous les rapports de risque se lisent *contre l'observation*. Si l'ordre était alphabétique, la référence deviendrait `Lev` et toute l'interprétation basculerait sans que rien ne signale l'erreur.

```
r  
  
sex      = factor(sex,      labels = c("Female", "Male")),  
obstruct = factor(obstruct, labels = c("No", "Yes")),  
perfor   = factor(perfor,  labels = c("No", "Yes")),  
adhere   = factor(adhere,  labels = c("No", "Yes")),  
differ   = factor(differ,  labels = c("Well", "Moderate", "Poor")),  
extent   = factor(extent,  labels = c("Submucosa", "Muscle", "Serosa", "Ad  
node4    = factor(node4,   labels = c("<=4 nodes", ">4 nodes")),  
surg     = factor(surg,    labels = c("Short", "Long"))  
)
```

Conversion des codes numériques en modalités lisibles. Ces libellés se propagent automatiquement dans les légendes, le forest plot et les tableaux de sortie.

Piège à connaître. `factor(x, labels = ...)` attribue les étiquettes **dans l'ordre trié des valeurs**, sans vérifier quoi que ce soit. Si le codage était inversé (`sex`: 1 = femme), les étiquettes seraient muettement fausses et tout le rapport aussi. Le contrôle se fait avec `table(colon$sex)` avant conversion — à faire une fois, puis à documenter.

```
r  
  
dim(df)  
summary(df$rx)
```

Contrôles de cohérence : on doit lire **929** lignes, et trois effectifs de bras. C'est la vérification que le filtre `etype` a bien opéré.

4. Chunk `descriptive` — description avant modélisation

```
r  
  
ggplot(df, aes(x = time / 365.25)) +  
  geom_histogram(bins = 30, ...)
```

`time` est en jours ; la division par 365,25 le convertit en années (la fraction tient compte des années bissextiles).

Pourquoi cette figure. La distribution du suivi révèle la structure de censure. Un essai clos à date fixe produit une censure administrative visible : un amas de patients censurés autour de la date de gel de la base. Savoir où le suivi devient clairsemé indique jusqu'où les courbes de Kaplan-Meier sont interprétables.

```
r  
  
df %>%  
  group_by(rx) %>%  
  summarise(n = n(), deaths = sum(status), event_rate = round(mean(status), 3))
```

`status` vaut 1 en cas de décès, 0 si censuré. Donc `sum(status)` = nombre d'événements et `mean(status)` = proportion d'événements observés.

Pourquoi c'est indispensable. Un lecteur veut les effectifs bruts avant de regarder une courbe. Une différence de survie portée par 12 événements ne se lit pas comme une différence portée par 200.

⚠ `event_rate` n'est **pas** un taux d'incidence : il ignore le temps d'exposition. C'est une proportion d'événements observés, sensible à la durée de suivi de chaque bras. À ne pas présenter comme un taux.

5. Chunk `km_overall` — courbe de survie globale

```
r  
  
fit_overall <- survfit(Surv(time, status) ~ 1, data = df)
```

`Surv(time, status)` construit l'**objet de survie** : chaque observation devient un couple (durée, indicateur d'événement). C'est la structure qui permet de traiter la censure à droite — un patient censuré à 3 ans apporte l'information « vivant jusqu'à 3 ans », sans qu'on sache ce qui suit.

`~ 1` signifie « aucun groupe » : une seule courbe pour l'ensemble.

`survfit` calcule l'estimateur de **Kaplan-Meier**, dit produit-limite :

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right)$$

où d_i est le nombre de décès au temps t_i et n_i le nombre de sujets encore à risque juste avant. À chaque événement, la survie est multipliée par la proportion de survivants du moment.

Hypothèse implicite : censure non informative. On suppose que les patients censurés ont le même pronostic que ceux qui restent suivis. Si les patients les plus graves sortaient de l'étude préférentiellement, la survie serait surestimée.

```
r
conf.int = TRUE
```

Bandes de confiance à 95 %, calculées par défaut sur l'échelle log-log — plus stable que l'échelle brute aux extrémités, où l'estimateur est très incertain.

Fonction dans le rapport : établir la forme de référence avant toute stratification.

6. Chunk `km_treatment` — survie par bras

```
r
fit_treatment <- survfit(Surv(time, status) ~ rx, data = df)
```

`~ rx` produit une courbe par bras.

```
r
pval = TRUE
```

Affiche la p-value du **test du log-rank global à trois groupes**.

Ce que teste le log-rank : H_0 = les fonctions de survie sont identiques dans tous les groupes. Il compare, à chaque temps d'événement, le nombre de décès observés dans chaque groupe au nombre attendu sous H_0 , et agrège. Il pondère tous les temps de la même façon — il est donc puissant quand les risques sont proportionnels, et faible si les courbes se croisent.

Limite à connaître : un test global significatif dit qu'*au moins un* bras diffère. Il ne dit pas lequel. D'où le chunk suivant.

```
r
risk.table = TRUE
```

Table du nombre de sujets à risque sous la courbe. **Élément non décoratif** : c'est ce qui permet de juger où la courbe cesse d'être fiable. Une chute apparente en fin de suivi portée par 8 patients à risque n'a pas la même valeur qu'une séparation à 2 ans portée par 250.

```
r  
  
palette = c("#888888", "#3498DB", "#1F3A5F")
```

Gris pour l'observation, deux tons de bleu pour les bras actifs : le codage couleur porte l'information « contrôle vs traitement ».

7. Chunk `logrank_pairwise` — contrastes par paire (ajouté)

```
r  
  
pairwise_logrank <- function(arm) {  
  sub <- df %>%  
    filter(rx %in% c("Obs", arm)) %>%  
    mutate(rx = droplevels(rx))
```

Sous-échantillon à deux bras.

`droplevels()` est **obligatoire** : sans lui, le facteur conserve son troisième niveau, désormais vide. `survdiff` compterait alors trois groupes, et les degrés de liberté du test seraient faux.

```
r  
  
lr <- survdiff(Surv(time, status) ~ rx, data = sub)  
p <- 1 - pchisq(lr$chisq, df = 1)
```

`survdiff` renvoie une statistique de khi-deux. Avec deux groupes, elle suit un χ^2 à **1 degré de liberté** sous H_0 ; `1 - pchisq()` donne la p-value unilatérale supérieure.

```
r  
  
format.pval(p, digits = 3, eps = 1e-4)
```

`eps` fixe le seuil sous lequel on affiche `<1e-04` au lieu d'une notation scientifique illisible.

Pourquoi ce chunk existe. C'est la question clinique de l'essai : le bénéfice vient-il de la combinaison ou du lévamisole seul ? Le lévamisole en monothérapie s'est révélé inefficace, et l'essai a fait de l'association 5-FU + lévamisole le standard du stade III. Conclure « le bras Lev+5FU montre un avantage » à partir du seul test global est une sur-interprétation.

Note : deux comparaisons planifiées contre un comparateur commun ne rendent pas la correction de multiplicité indispensable. Si tu veux être strict, `p.adjust(method = "holm")`,

et tu le mentionnes.

8. Chunk `cox_univariate` — effet brut du traitement

```
r  
  
cox_uni <- coxph(Surv(time, status) ~ rx, data = df)  
summary(cox_uni)
```

Le modèle de Cox à risques proportionnels :

$$h(t | X) = h_0(t) \exp(\beta_1 X_1 + \dots + \beta_p X_p)$$

- $h(t)$: le **hasard**, risque instantané de décès à l'instant t sachant qu'on y est parvenu vivant
- $h_0(t)$: hasard de base, **jamais estimé** — c'est ce qui rend le modèle *semi-paramétrique*
- $\exp(\beta)$: le **rapport de risque (HR)**, supposé constant dans le temps

L'estimation passe par la **vraisemblance partielle** : à chaque temps d'événement, on ne modélise que la probabilité que ce soit cet individu-là qui subisse l'événement parmi ceux à risque. Le hasard de base s'annule dans le rapport, d'où le fait qu'on n'ait pas besoin de le spécifier.

Rôle de ce modèle non ajusté : fournir un point de comparaison. Si l'estimation ajustée s'en écarte fortement, il y a confusion ; si elle en reste proche, la randomisation a équilibré les facteurs mesurés.

HR ≠ risque relatif. Un HR de 0,69 signifie qu'à tout instant, parmi les sujets encore vivants, le risque instantané de décès est 31 % plus bas dans le bras traité. Cela ne dit pas que 31 % de décès en moins surviendront sur la durée : la traduction en survie absolue dépend du hasard de base.

9. Chunk `cox_multivariate` — effet ajusté

```
r  
  
cox_multi <- coxph(Surv(time, status) ~ rx + age + sex + obstruct +  
                  perfor + adhere + differ + extent + node4 + surg,  
                  data = df)
```

Deux objectifs distincts :

1. **Estimer l'effet du traitement ajusté** sur les facteurs pronostiques connus. Dans un essai randomisé, ce n'est pas une correction de confusion mais un **gain de précision** : retirer la variabilité expliquée par le pronostic resserre l'intervalle de confiance.
2. **Estimer les facteurs pronostiques eux-mêmes**, qui ont un intérêt propre.

Pourquoi `nodes` est absent. `node4` est la dichotomisation de `nodes` (> 4 ganglions envahis). Inclure les deux introduirait une colinéarité structurelle qui gonflerait les erreurs standards sur deux des termes les plus prédictifs du modèle. Choix assumé et documenté.

`age` est traité comme linéaire sur l'échelle du log-hasard. Hypothèse forte, non testée ici : une spline ou un polynôme fractionnaire seraient l'alternative. Figure dans les limitations.

Ce qui se passe silencieusement : `coxph` supprime toute ligne comportant une covariable manquante (*listwise deletion*). L'effectif du modèle est donc inférieur à 929, sans aucun message.

10. Chunk `missing_data_check` — exposer la suppression (ajouté)

```
r

cat("Univariate model - n =", cox_uni$n, ", events =", cox_uni$nevent, "\n")
cat("Multivariate model - n =", cox_multi$n, ", events =", cox_multi$nevent, "\n")
cat("Rows dropped due to missing covariates:", nrow(df) - cox_multi$n, "\n\n")
```

`$n` et `$nevent` sont des composants de l'objet `coxph` : effectif effectivement utilisé et nombre d'événements observés.

```
r

covars    <- all.vars(formula(cox_multi))[-(1:2)]
na_counts <- colSums(is.na(df[, covars, drop = FALSE]))
print(na_counts[na_counts > 0])
```

`all.vars(formula(...))` extrait tous les noms de variables de la formule. `[-(1:2)]` retire les deux premiers, `time` et `status`, laissant les covariables. `colSums(is.na(...))` compte les manquants par variable, et le filtre final n'affiche que celles concernées.

Pourquoi laisser le code nommer la variable. Écrire de mémoire « les NA viennent de `differ` et `node4` » est une inexactitude vérifiable en une ligne : `node4` est complète. Une affirmation fausse dans un README de démonstration méthodologique coûte plus qu'elle ne rapporte.

11. Chunk `sensitivity_missing` — analyse de sensibilité (ajouté)

```
r

cox_nodiffer <- coxph(Surv(time, status) ~ rx + age + sex + obstruct +
                      perfor + adhere + extent + node4 + surg, data = df)
```

Même modèle, `(differ)` retiré. Comme c'est la seule source de manquants, ce modèle conserve les **929** patients.

```
r  
  
data.frame(  
  model = c("With differ (complete cases)", "Without differ (all patients)"),  
  n      = c(cox_multi$n, cox_nodiffer$n),  
  HR     = round(c(summary(cox_multi)$conf.int["rxLev+5FU", "exp(coef)"],  
                  summary(cox_nodiffer)$conf.int["rxLev+5FU", "exp(coef)"]), 3)  
)
```

Logique. Si l'estimation de traitement bouge peu entre les deux, la suppression listwise n'introduit pas de biais matériel — et c'est *démontré*, non supposé. Si elle bouge, il faut le dire et envisager une imputation.

Ce qu'il ne faut jamais faire : compléter les `(differ)` manquants à partir de la publication de Moertel. L'article ne publie que des statistiques agrégées ; aucune valeur individuelle ne s'en déduit. Ce serait de la fabrication de données — l'erreur qui invalide tout le reste dans un projet dont l'objet déclaré est la rigueur.

Ce qu'on ferait proprement si l'écart était important : imputation multiple (`(mice)`). En survie, il faut inclure l'indicateur d'événement **et** le hasard cumulé de Nelson-Aalen dans le modèle d'imputation, faute de quoi l'estimation de traitement est biaisée vers l'absence d'effet.

12. Chunk `(forest_plot)`

```
r  
  
ggforest(cox_multi, data = df)
```

Représente HR et intervalle de confiance pour chaque terme du modèle, sur échelle logarithmique. La ligne verticale à 1 marque l'absence d'effet : un intervalle qui la franchit correspond à un résultat non significatif au seuil de 5 %.

Réserve à connaître. `(ggforest)` lit les effectifs par modalité dans `(df)`, alors que le modèle a été ajusté sur les cas complets. Les N affichés sont donc légèrement supérieurs aux N réels. Les HR et intervalles, eux, restent corrects. Deux traitements possibles : le signaler en légende, ou passer `(data = model.frame(cox_multi))`.

13. Chunks `(ph_test)` et `(ph_plot)` — vérification de l'hypothèse

```
r
```

```
ph_test <- cox.zph(cox_multi)
ph_test
ph_global_p <- ph_test$table["GLOBAL", "p"]
```

Ce qui est testé. Le modèle de Cox suppose que les HR sont **constants dans le temps**.

`cox.zph` s'appuie sur les **résidus de Schoenfeld** : à chaque temps d'événement et pour chaque covariable, l'écart entre la valeur observée chez le sujet qui subit l'événement et la moyenne pondérée des sujets encore à risque.

Sous l'hypothèse de proportionnalité, ces résidus mis à l'échelle n'ont **aucune tendance en fonction du temps**. `cox.zph` teste la nullité de cette pente, covariable par covariable, plus un test global.

```
r
ggcoxzph(ph_test) # fig.height = 10
```

Un panneau par covariable : résidus contre temps, avec une courbe lissée. Une courbe plate valide visuellement l'hypothèse ; une pente marquée signale un effet dépendant du temps.

`fig.height = 10` parce qu'il y a une dizaine de panneaux à empiler.

Que faire en cas de violation. Trois options, par ordre de complexité : stratifier sur la covariable fautive (`strata()`), qui lui laisse son propre hasard de base sans estimer de HR), introduire une interaction avec le temps, ou restreindre l'analyse à une fenêtre où la proportionnalité tient.

Point stratégique. Une violation détectée, traitée et rapportée vaut *mieux* qu'un silence. C'est ce qui distingue une analyse conduite d'une analyse récitée.

14. Section `# Interpretation` — R inline

Le paragraphe utilise du R inline (``r ...``) plutôt que des valeurs saisies à la main. **Raison** : une valeur écrite en dur peut diverger du modèle après n'importe quelle modification du code, sans que rien ne le signale. Le R inline rend la dérive impossible.

```
r
`r if (ph_global_p >= 0.05) sprintf(...) else sprintf(...)`
```

La condition **remplace** la phrase entière plutôt que de lui accoler une réserve. Une formulation du type « l'hypothèse est vérifiée ($p = 0,01$) — bien qu'elle puisse être violée » est auto-contradictoire : l'affirmation principale dit l'inverse du test qu'elle cite.

Avant de knitter : vérifier `rownames(summary(cox_multi)$coefficients)`. Si le nom du coefficient n'est pas exactement `"rxLev+5FU"`, le rendu échoue à la dernière étape et la compilation entière est perdue.

15. Chunks `readme_numbers` et `session`

`readme_numbers` regroupe en une sortie unique tous les nombres que le README attend. Le README est du markdown **statique** : il n'exécute pas de R, contrairement au rapport. Les deux mécanismes sont donc complémentaires, pas concurrents.

`sessionInfo()` enregistre versions de R, des paquets, système d'exploitation et locale. C'est ce qui permet à un tiers de reconstituer l'environnement exact — le complément indispensable de `renv.lock`, qui fige les paquets mais pas R lui-même.

Questions d'oral probables

Classées par probabilité décroissante. Les trois premières tomberont presque certainement.

« Pourquoi avoir filtré sur `etype == 2` ? »

Le jeu contient deux lignes par patient — récidive et décès. Sans filtre, chaque patient compte double : les erreurs standards sont sous-estimées et les p-values artificiellement petites, sans le moindre avertissement. J'ai retenu la survie globale comme critère de jugement.

« Qu'est-ce qu'un rapport de risque, exactement ? »

Le rapport de deux risques instantanés. Un HR de 0,69 signifie qu'à chaque instant, parmi les sujets encore vivants, le risque immédiat de décès est 31 % plus faible dans le bras traité. Ce n'est pas un risque relatif : la traduction en survie absolue dépend du hasard de base, que le modèle de Cox n'estime pas.

« Comment avez-vous vérifié l'hypothèse de proportionnalité ? »

Par les résidus de Schoenfeld, via `cox.zph`, avec un test global et un test par covariable, plus une inspection graphique des résidus contre le temps. (*Puis : le résultat obtenu, et ce que tu en as fait.*)

« Pourquoi le modèle de Cox et pas une régression logistique ? »

La régression logistique modélise une issue binaire à horizon fixe et n'exploite ni la date de survenue ni la censure. Ici, un patient censuré à 3 ans apporte une information réelle — vivant jusqu'à 3 ans — que la logistique jetterait ou traiterai à tort comme un survivant définitif.

« Pourquoi dit-on que le modèle de Cox est semi-paramétrique ? »

Parce que l'effet des covariables est paramétrique — un coefficient par variable — tandis que le hasard de base reste totalement non spécifié. La vraisemblance partielle permet d'estimer les coefficients sans jamais l'estimer, puisqu'il s'annule dans le rapport.

« Que teste précisément le log-rank ? »

L'égalité des fonctions de survie entre groupes. Il compare, à chaque temps d'événement, les décès observés aux décès attendus sous cette hypothèse. Il pondère uniformément tous les temps de suivi : puissant sous proportionnalité, faible si les courbes se croisent.

« **Votre test global est significatif — que pouvez-vous en conclure ?** »

Qu'au moins un bras diffère, sans savoir lequel. C'est pourquoi j'ai ajouté des contrastes par paire contre l'observation : ils montrent que le bénéfice est porté par la combinaison, le lévamisole seul ne se séparant pas du contrôle.

« **Pourquoi avoir exclu `nodes` ?** »

`node4` en est la dichotomisation. Inclure les deux créerait une colinéarité structurelle qui gonflerait les erreurs standards sur deux des variables les plus pronostiques du modèle.

« **Comment avez-vous traité les données manquantes ?** »

Analyse en cas complets, avec deux garde-fous : je documente combien de lignes sont supprimées et par quelle variable, et je refais tourner le modèle sans la variable concernée pour vérifier que l'estimation de traitement ne bouge pas. Sous hypothèse MAR, l'imputation multiple serait plus rigoureuse ; en survie elle exige d'inclure l'indicateur d'événement et le hasard cumulé de Nelson-Aalen dans le modèle d'imputation, ce que j'ai jugé hors périmètre et signalé comme tel.

« **Vous parlez de risque compétitif — de quoi s'agit-il ?** »

Pour un critère de récurrence, un décès sans récurrence empêche définitivement d'observer l'événement. Le traiter comme une censure reviendrait à supposer que le patient aurait pu récidiver plus tard, ce qui est faux. Il faudrait alors une incidence cumulée avec estimateur d'Aalen-Johansen, ou un modèle de Fine-Gray. En me restreignant à la survie globale, je contourne le problème plutôt que je ne le résous — c'est indiqué dans les limitations.

« **Qu'apporte la table du nombre de sujets à risque ?** »

Elle indique où la courbe cesse d'être fiable. Une séparation en fin de suivi portée par une poignée de patients n'a pas le même poids qu'une séparation précoce portée par plusieurs centaines.

« **Vos résultats sont-ils cliniquement transposables ?** »

Non. Le standard de 1990 a été remplacé par les protocoles à base d'oxaliplatine. Le projet démontre une méthode sur un jeu de référence ; il ne produit aucun résultat clinique nouveau. Le seul intérêt de la comparaison aux 33 % publiés est de vérifier que mon pipeline se comporte comme attendu.

« **Que feriez-vous ensuite ?** »

Trois extensions, par coût croissant : indice de concordance et calibration avec correction bootstrap de l'optimisme ; Cox pénalisé pour la sélection de variables ; forêt de survie aléatoire en comparaison, pour confronter un modèle statistique et un modèle d'apprentissage sur le même problème.

Le piège de l'oral

La question la plus déstabilisante n'est pas technique, c'est : « **qu'est-ce que ce projet démontre, au juste ?** »

La réponse honnête, et la meilleure : une maîtrise méthodologique sur un jeu de données de référence, pas une contribution scientifique. Le jeu est un classique d'enseignement, le résultat est connu depuis 1990, et le standard thérapeutique a changé depuis. Ce que le projet montre, c'est la capacité à définir un critère de jugement, à repérer les pièges de structure des données, à vérifier une hypothèse de modèle plutôt qu'à la supposer, et à livrer un travail que quelqu'un d'autre peut reproduire.

Revendiquer davantage se retourne immédiatement. Revendiquer exactement cela est solide.